Mizan-Tepi University

School of Computing & Informatics
Department of IT

Course Title: Data Structures and Algorithms

Course Code:ITec2052

Group Assignment – **Prepared by GROUP 5**

Year/Semester: **2nd Year 2nd Semester**

Enrolment: **Regular** (Degree Program)

S/N	Students' Names	ID No.
1.	Mut Tut	MTUUR/4493/16
2.	Alazer Agumas	MTUUR/4041/16
3.	Samson Obang	MTUUR/4553/16
4.	Tefera Mitiku	MTUUR/4614/16
5.	Melaku Bayingi	MTUUR/4439/16

Submitted to Instructor: **Mr. Chalewe Z. MSc**

Submission date: **31 May, 2025 G.C**

Mizan-Tepi, Ethiopia

Pharmacy Medicine Inventory Management System



Pharmacy Medicine Inventory Management System.



ABSTRACT

"Pharmacy is the backbone of any healthcare system, providing vital medications and services to ensure the well-being of the community." In order to maintain this critical role effectively, a reliable and efficient system is needed to manage medicine inventory.

This project, titled **Pharmacy Medicine Inventory Management System**, is a software solution developed in **C++** using the **Linked List** data structure to manage the dynamic nature of pharmacy inventory. It is designed to help pharmacists and staff keep track of available medicines, monitor expiry dates, and ensure that only safe, non-expired medicines are in stock.

The system includes core functionalities such as:

- **Adding new medicines**
- **Deleting expired medicines**
- **Searching medicines by name**

Each medicine is represented as a node in a **singly linked list**, allowing efficient insertion, deletion, and traversal. This data structure is particularly suitable for inventory management due to its flexibility in handling an unknown or changing number of records without memory waste.

The project ensures smooth inventory operations, enhances medication safety, and contributes to improved healthcare delivery by maintaining an up-to-date and searchable medicine database.

Acknowledgement

We would like to express our sincere gratitude and appreciation to **Mr. Chalewe Zeynu** for his invaluable guidance, support, and encouragement throughout the development of this project titled **Pharmacy Medicine Inventory Management System**.

His expertise, motivation, and constructive feedback have played a crucial role in helping us understand the concepts of data structures, particularly **linked lists**, and how they can be applied in real-world systems.

This project would not have been possible without his dedication and commitment to student success.

Table of Contents

ABSTRACT i

🙏 Acknowledgement ii

Introduction v

Background: Challenges of Manual Inventory Management 1

Emergence of Automated Solutions 1

Significance of the Project 1

Technical Tools 2

Programming Language 2

Development Environment 2

Data Storage 2

System Overview 2

How It Works 3

✓ Data Structure: Linked List 4

1. Insertion (Adding Medicine) 5

2. □ Deletion 8

💊 Deleting Expired Medicine 8

📄 Description 8

📌 Algorithm 8

3. Search 11

🔍 Searching Medicine 11

Description 11

Algorithm 11

Conclusion 18

References 19

Introduction

Inventory management in a pharmacy ensures that the right medicines are available at the right time, in the correct quantities. Traditional manual inventory practices are inefficient, leading to stockouts, overstocking, and expired medicines. This project introduces an **automated Pharmacy Medicine Inventory Management System** developed using **C++ and a linked list** to provide a dynamic, efficient solution for managing medicine inventory.

> Inventory management is a critical component in the efficient operation of pharmacy stores. The manual approach to inventory tracking is often time-consuming, error-prone, and inefficient. In this project, we introduce an automated Pharmacy Medicine Inventory Management System using C++ and the linked list data structure.

The **Pharmacy Medicine Inventory Management System** is a software solution designed to efficiently manage the stock of medicines in a pharmacy. This system helps pharmacy staff to:

- **Add new medicines** to the inventory.
- **Remove expired medicines** from the inventory to maintain up-to-date stock.
- **Search for medicines** by name to quickly find specific products.

This system utilizes a **linked list** to store medicine information, such as:

- **Name** of the medicine.
- **Quantity** in stock.
- **Category** (e.g., antibiotic, painkiller).
- **Expiry date** to ensure medicines are not outdated.

Pharmacy Medicine Inventory Management System

Background: Challenges of Manual Inventory Management

Description:
Manual systems rely on paper logs or basic spreadsheets, which are not scalable or reliable for growing pharmacy operations.

Manual inventory management in pharmacies presents several challenges:
- **Prone to human errors** (e.g., incorrect entries, misplacing records)
- **Difficulty in tracking expired medicines**, leading to legal and health hazards
- **Time-consuming to search** for a specific medicine
- **Inefficient stock updates and reporting**, making restocking decisions harder

Emergence of Automated Solutions

Description:
Software systems reduce the dependency on human input and automate critical tasks.

To overcome these issues, software-based inventory systems have emerged, offering:
- **Real-time updates** for inventory status
- **Automated expiry tracking** to flag or remove expired products
- **Faster and more accurate search** functionality using optimized data structures
- **Easier data manipulation** (insert, delete, update), allowing scalability

Significance of the Project

Description:
The system aims to provide pharmacy staff with an intuitive tool to manage stock levels, reduce waste, and improve service delivery.

This project aims to develop a simple yet efficient inventory system for pharmacies that:
- Keeps track of available medicines
- Allows easy addition and removal of medicines
- Searches medicines quickly by name
- Helps maintain up-to-date inventory

Technical Tools

Programming Language

Description:

C++ is chosen for its performance, memory control, and suitability for implementing **low-level data structures like linked lists.**



• C++: Chosen for its efficiency and support for object-oriented programming, as well as its handling of dynamic memory and data structures.

Development Environment

Description:

Popular IDEs were used to simplify development with features such as debugging and autocomplete.

• IDE: Code::Blocks / Visual Studio Code, 

DEV++

Features used:

- Syntax highlighting
- Code completion
- Integrated debugger

Data Storage

- In-memory storage using linked list
- Each medicine is a node in the list
- Attributes: name, quantity, expiry date

System Overview

Description:

A linked list-based inventory system that supports three core operations—insert, delete, and search—allowing efficient dynamic memory usage and ease of updates.

This system uses linked lists to manage pharmacy inventory efficiently. It includes core operations:

• **Insertion** (adding medicines)
• **Deletion** (removing expired or unnecessary stock)
• **Search** (finding a medicine by name)

How It Works

Linked List Structure:

The medicines are stored in a **linked list**, with each node representing a medicine. Each node contains:

- **Name** (string)
- **Quantity** (integer)
- **Category** (string)
- **Expiry Date** (string in YYYY-MM-DD format)
- **Next Pointer** (to link to the next medicine node)

Operations:

- **Insert**: New medicines are added at the end of the list.
- **Delete Expired**: Expired medicines are removed from the list by comparing their expiry date with the current date.
- **Search**: Medicines are searched by name, and their details are displayed if found.

1. Data Structure: Linked List

Definition:

A linked list is a linear data structure in which each element (node) points to the next. It allows **dynamic memory allocation** and **efficient insertion/deletion** without reallocating or reorganizing memory.

In this project, the primary data structure used is the **Linked List**.

A **Linked List** is a dynamic data structure consisting of **nodes**, where each node contains:

- **Medicine Name**
- **Quantity in Stock**
- **Category**
- **Expiry Date**
- A **Pointer to the Next Node**

This structure is ideal for a pharmacy inventory system because:

- It allows **efficient insertion and deletion** of medicine records without shifting elements (unlike arrays).
- It uses **dynamic memory allocation**, meaning it grows or shrinks as needed based on the number of medicines.
- It supports **easy traversal** for searching and checking expiry dates.

Each medicine is stored as a node in a **singly linked list**, connected in sequence using pointers.

This enables smooth and fast operations like:

- **Adding new medicines** at the end.
- **Deleting expired medicines** by comparing dates and updating links.
- **Searching** by traversing nodes based on the medicine name.

C++ Code:

```
struct Medicine {
   string name;
   int quantity;
   string expiry;
   Medicine* next;
};
```

Why Linked List?

- **Efficient Insertion and Deletion**: Inserting or deleting a medicine doesn't require shifting all elements (as in an array). The linked list dynamically adjusts the pointers for the insertion or deletion.
- **Dynamic Memory Allocation**: The list can grow or shrink depending on the number of medicines in the inventory without allocating fixed-size memory.
- **Easy Traversal**: The system can easily traverse the list to perform tasks like searching and checking expiry dates.

1. Insertion (Adding Medicine)

Definition:
Adds a new medicine node at the end of the linked list. It involves creating a new node and linking it to the list.

■ Description:

This feature enables users to add a new medicine to the inventory, which is implemented as a **singly linked list**. Each medicine is stored as a node containing the medicine's **name**, **quantity**, **category**, and **expiry date**. When a new medicine is entered, a new node is dynamically created and appended to the end of the list. This structure supports **dynamic memory allocation**, which is more flexible than static arrays and prevents memory waste.

□ *Algorithm:*

1. **Input Medicine Details**
   - Prompt the user to enter:
     - Name of the medicine
     - Quantity
     - Category (e.g., antibiotic, painkiller)
     - Expiry date
2. **Create a New Node**
   - Dynamically allocate memory for a new `Medicine` node.
   - Assign all entered values to the node's attributes.
3. **Check for Empty List**
   - If the list is empty (`head == nullptr`), make the new node the head of the list.
4. **Traverse to the End**
   - If the list is not empty, traverse to the last node (`next == nullptr`).
5. **Link the New Node**
   - Set the `next` pointer of the last node to point to the new medicine node.
6. **Save to File (Optional)**
   - You can implement a `saveToFile()` function to save the list contents to disk for persistence.
7. **Display Success Message**
   - Notify the user that the medicine was successfully added.

□ *Time Complexity:*

- **Best Case**: O(1) – If the list is empty, the new node becomes the head.
- **Worst Case**: O(n) – If the list has many elements, traversal to the end is required.

C++ Code:

```
#include <iostream>
#include <string>

using namespace std;

// Define the Medicine structure
struct Medicine {
    string name;
    int quantity;
    string category;
    string expiry;
    Medicine* next;
};

// Function to insert a new medicine into the inventory
void insertMedicine(Medicine*& head) {
    Medicine* newMed = new Medicine;

    cout << "Enter medicine name: ";
    cin >> newMed->name;
    cout << "Enter quantity: ";
    cin >> newMed->quantity;
    cout << "Enter category: ";
    cin >> newMed->category;
    cout << "Enter expiry date (YYYY-MM-DD): ";
    cin >> newMed->expiry;

    newMed->next = nullptr;

    if (head == nullptr) {
        head = newMed;
    } else {
        Medicine* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newMed;
    }

    cout << "□ Medicine successfully added to inventory." << endl;
}

// Function to display all medicines in the inventory
void displayMedicines(Medicine* head) {
    if (head == nullptr) {
        cout << "No medicines in the inventory." << endl;
        return;
    }
```

```
    Medicine* temp = head;
    while (temp != nullptr) {
        cout << "Name: " << temp->name << ", Quantity: " << temp->quantity
             << ", Category: " << temp->category << ", Expiry: " << temp->expiry << endl;
        temp = temp->next;
    }
}

int main() {
    Medicine* inventory = nullptr;
    int choice;

    do {
        cout << "\nInventory Menu:\n";
        cout << "1. Add Medicine\n";
        cout << "2. Display Medicines\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                insertMedicine(inventory);
                break;
            case 2:
                displayMedicines(inventory);
                break;
            case 3:
                cout << "Exiting the program.\n";
                break;
            default:
                cout << "Invalid choice! Please try again.\n";
        }
    } while (choice != 3);

    // Clean up memory before exiting
    Medicine* temp;
    while (inventory != nullptr) {
        temp = inventory;
        inventory = inventory->next;
        delete temp;
    }

    return 0;
}
```

2. 🗑 Deletion

Definition:
Removes a medicine node by name. It traverses the list, finds the node, adjusts pointers, and deletes the memory.

💊 Deleting Expired Medicine

📄 Description

This feature deletes medicines from the inventory that have expired. The user provides the current date, and the system removes all medicines with expiry dates earlier than the current date.

📌 Algorithm

1. **Input Current Date**
   - Prompt the user to enter the current date in `YYYY-MM-DD` format.
2. **Initialize Stack**
   - Declare a stack to temporarily store **non-expired medicines**.
3. **Compare Dates**
   - Iterate through the linked list of medicines and compare each medicine's expiry date with the current date.
   - If the medicine has expired, display a message and **exclude** it from the new inventory list.
4. **Update Inventory**
   - Transfer non-expired medicines from the stack back to the linked list.
5. **Save to File**
   - Call `saveMedicinesToFile()` to save the updated inventory to a file.
6. **Confirmation**
   - Display a success message indicating that expired medicines have been deleted.

💻 C++ Code

```
#include <iostream>
#include <string>
using namespace std;

// Define the Medicine structure
struct Medicine {
    string name;
    int quantity;
    string category;
    string expiry;
    Medicine* next;
};

// Function to insert a new medicine into the inventory
```

```
void insertMedicine(Medicine*& head) {
    Medicine* newMed = new Medicine;

    cout << "Enter medicine name: ";
    cin >> newMed->name;
    cout << "Enter quantity: ";
    cin >> newMed->quantity;
    cout << "Enter category: ";
    cin >> newMed->category;
    cout << "Enter expiry date (YYYY-MM-DD): ";
    cin >> newMed->expiry;

    newMed->next = nullptr;

    if (head == nullptr) {
        head = newMed;
    } else {
        Medicine* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newMed;
    }

    cout << "Medicine successfully added to inventory." << endl;
}

// Function to display all medicines in the inventory
void displayMedicines(Medicine* head) {
    if (head == nullptr) {
        cout << "No medicines in the inventory." << endl;
        return;
    }
    Medicine* temp = head;
    while (temp != nullptr) {
        cout << "Name: " << temp->name << ", Quantity: " << temp->quantity
             << ", Category: " << temp->category << ", Expiry: " << temp-
>expiry << endl;
        temp = temp->next;
    }
}

// Function to delete expired medicines from the inventory
void deleteExpiredMedicines(Medicine*& head) {
    string currentDate;
    cout << "Enter current date (YYYY-MM-DD): ";
    cin >> currentDate;

    Medicine* temp = head;
    Medicine* prev = nullptr;

    while (temp != nullptr) {
        if (temp->expiry < currentDate) {
            cout << "Deleted expired medicine: " << temp->name << endl;

            if (prev == nullptr) {
```

```
                head = temp->next;
            } else {
                prev->next = temp->next;
            }

            Medicine* toDelete = temp;
            temp = temp->next;
            delete toDelete;
        } else {
            prev = temp;
            temp = temp->next;
        }
    }

    cout << "□ Expired medicines successfully deleted." << endl;
}

int main() {
    Medicine* inventory = nullptr;
    int choice;

    do {
        cout << "\nInventory Menu:\n";
        cout << "1. Add Medicine\n";
        cout << "2. Display Medicines\n";
        cout << "3. Delete Expired Medicines\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                insertMedicine(inventory);
                break;
            case 2:
                displayMedicines(inventory);
                break;
            case 3:
                deleteExpiredMedicines(inventory);
                break;
            case 4:
                cout << "Exiting the program.\n";
                break;
            default:
                cout << "Invalid choice! Please try again.\n";
        }
    } while (choice != 4);
    // Clean up memory before exiting
    Medicine* temp;
    while (inventory != nullptr) {
        temp = inventory;
        inventory = inventory->next;
        delete temp;
    }
    return 0;
}
```

Time Complexity:

- **Best Case:** O(1) (if first node)
- **Worst Case:** O(n) (traverse to find the medicine)

Use Case: When a medicine expires or is discontinued.

3. Search

Definition:
Finds and displays a medicine's details by comparing names node-by-node.

🔍 Searching Medicine

📄 Description

This feature allows the user to search for a medicine by its name. If the medicine is found, the system displays its details.

📌 Algorithm

1. **Input Medicine Name**
   - Prompt the user to enter the medicine name to search for.
2. **Search for Medicine**
   - Iterate through the linked list to find the medicine with the specified name.
3. **Display Details**
   - If the medicine is found, display its name, quantity, category, and expiry date.
4. **Error Handling**
   - If the medicine is not found, display an error message.

💻 C++ Code

```
#include <iostream>

#include <string>

using namespace std;

// Define the Medicine structure

struct Medicine {

    string name;
```

```
    int quantity;
    string category;
    string expiry;
    Medicine* next;
};
// Function to insert a new medicine into the inventory
void insertMedicine(Medicine*& head) {
    Medicine* newMed = new Medicine;
    cout << "Enter medicine name: ";
    cin >> newMed->name;
    cout << "Enter quantity: ";
    cin >> newMed->quantity;
    cout << "Enter category: ";
    cin >> newMed->category;
    cout << "Enter expiry date (YYYY-MM-DD): ";
    cin >> newMed->expiry;
    newMed->next = nullptr;
    if (head == nullptr) {
        head = newMed;
    } else {
        Medicine* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
```

```
        }
        temp->next = newMed;
    }
    cout << "□ Medicine successfully added to inventory." <<
endl;
}
// Function to display all medicines in the inventory
void displayMedicines(Medicine* head) {
    if (head == nullptr) {
        cout << "No medicines in the inventory." << endl;
        return;
    }
    Medicine* temp = head;
    while (temp != nullptr) {
        cout << "Name: " << temp->name << ", Quantity: " <<
temp->quantity
             << ", Category: " << temp->category << ", Expiry: "
<< temp->expiry << endl;
        temp = temp->next;
    }
}
// Function to delete expired medicines from the inventory
void deleteExpiredMedicines(Medicine*& head) {
    string currentDate;
    cout << "Enter current date (YYYY-MM-DD): ";
```

```
    cin >> currentDate;
    Medicine* temp = head;
    Medicine* prev = nullptr;
    while (temp != nullptr) {
        if (temp->expiry < currentDate) {
            cout << "Deleted expired medicine: " << temp->name
<< endl;
            // If it's the first node
            if (prev == nullptr) {
                head = temp->next;
            } else {
                prev->next = temp->next;
            }
            Medicine* toDelete = temp;
            temp = temp->next;
            delete toDelete;
        } else {
            prev = temp;
            temp = temp->next;
        }
    }
    cout << "□ Expired medicines successfully deleted." << endl;
}
// Function to search for a medicine by its name
```

```
void searchMedicine(Medicine* head) {
    string searchName;
    cout << "Enter medicine name to search: ";
    cin >> searchName;
    Medicine* temp = head;
    bool found = false;
    while (temp != nullptr) {
        if (temp->name == searchName) {
            cout << "Medicine found!" << endl;
            cout << "Name: " << temp->name << endl;
            cout << "Quantity: " << temp->quantity << endl;
            cout << "Category: " << temp->category << endl;
            cout << "Expiry Date: " << temp->expiry << endl;
            found = true;
            break;
        }
        temp = temp->next;
    }

    if (!found) {
        cout << "□ Medicine not found in inventory." << endl;
    }
  }
int main() {
```

```
Medicine* inventory = nullptr;
int choice;

do {
    cout << "\nInventory Menu:\n";
    cout << "1. Add Medicine\n";
    cout << "2. Display Medicines\n";
    cout << "3. Delete Expired Medicines\n";
    cout << "4. Search Medicine\n";
    cout << "5. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
        case 1:
            insertMedicine(inventory);
            break;
        case 2:
            displayMedicines(inventory);
            break;
        case 3:
            deleteExpiredMedicines(inventory);
            break;
```

```
            case 4:
                searchMedicine(inventory);
                break;
            case 5:
                cout << "Exiting the program.\n";
                break;
            default:
                cout << "Invalid choice! Please try again.\n";
        }
    } while (choice != 5);
    // Clean up memory before exiting
    Medicine* temp;
    while (inventory != nullptr) {
        temp = inventory;
        inventory = inventory->next;
        delete temp;
    }
    return 0;
}
```

Time Complexity:

- **Best Case:** O(1) (first element is a match)
- **Worst Case:** O(n) (medicine is not present or last in list)
- **Use Case:** When a customer requests information or staff needs stock details.

Conclusion

Description:
This system demonstrates a simple yet powerful use of **linked lists in C++** for real-world applications like pharmacy inventory management. It ensures fast access, easy updates, and expandable architecture for future enhancements.

This Pharmacy Medicine Inventory Management System showcases the practical application of linked lists in C++. It allows for dynamic data handling, making inventory operations efficient and error-free. Future improvements can include file storage, GUI integration, and advanced sorting or filtering.

References

1. **Shah, P., & Patel, R. (2020).** "A Study on Sales and Inventory Management System for Pharmacy Stores." International Journal of Research in Computer Applications and Robotics.
2. GeeksforGeeks: Linked List Data Structure | GeeksforGeeks
3. Stack Overflow: c++ - how to correctly initialize a LinkList - Stack Overflow
4. YouTube Tutorial: https://youtu.be/6wXZ_m3SbEs?si=MFyaUZkqTuuD8LdA
5. W3Schools: DSA Linked Lists
6. Programiz: programiz.com
7. Google Search: Pharmacy Medicine Inventory Management System - Search
8. ResearchGate: 12283 PDFs | Review articles in DSA